

# CSE563: Unit I Study Notes

## Basic Mathematics in Computer Science

### School of Computer Science & Engineering

Session 2025–26

## Contents

|          |                                                            |          |
|----------|------------------------------------------------------------|----------|
| <b>1</b> | <b>☞ Unit I: Basic Mathematics in Computer Science</b>     | <b>2</b> |
| 1.1      | Number Theory . . . . .                                    | 2        |
| 1.1.1    | Divisibility and the Division Algorithm . . . . .          | 2        |
| 1.1.2    | Primes and the Fundamental Theorem of Arithmetic . . . . . | 2        |
| 1.1.3    | GCD, LCM and the Euclidean Algorithm . . . . .             | 3        |
| 1.1.4    | Modular Arithmetic and Congruences . . . . .               | 4        |
| 1.2      | Set Theory . . . . .                                       | 5        |
| 1.3      | Boolean Algebra . . . . .                                  | 6        |
| 1.3.1    | Canonical Forms . . . . .                                  | 7        |
| 1.4      | Logic . . . . .                                            | 7        |
| 1.4.1    | Propositional Logic . . . . .                              | 7        |
| 1.4.2    | Rules of Inference . . . . .                               | 8        |
| 1.4.3    | Predicate Logic and Quantifiers . . . . .                  | 8        |
| 1.5      | Calculus for Computer Science . . . . .                    | 9        |
| 1.6      | Mathematical Induction . . . . .                           | 10       |
| 1.7      | Relations and Functions . . . . .                          | 11       |
| 1.7.1    | Relations . . . . .                                        | 11       |
| 1.7.2    | Functions . . . . .                                        | 12       |
| 1.8      | Unit I Quick-Revision Sheet . . . . .                      | 13       |

# 1 Unit I: Basic Mathematics in Computer Science

**Course Outcome CO1:** recall the basic concepts of mathematics in computer science and engineering.

This unit is the toolbox for the rest of the course. Number theory drives hashing and cryptography; set theory and relations are the language of databases; Boolean algebra *is* the hardware; logic is how we specify and prove programs; calculus is the engine under every optimiser in machine learning; and induction is how we prove that recursive algorithms are correct.

## Map of the Unit

| Topic                 | Core idea                            | Used in CS for                      |
|-----------------------|--------------------------------------|-------------------------------------|
| Number theory         | Divisibility, primes, mod arithmetic | Hashing, RSA, random numbers        |
| Set theory            | Collections and their operations     | SQL, type systems, probability      |
| Boolean algebra       | Algebra of $\{0, 1\}$                | Digital circuits, conditionals      |
| Logic                 | Propositions, predicates, inference  | Verification, AI, SQL WHERE         |
| Calculus              | Rates of change, accumulation        | Gradient descent, signal processing |
| Induction             | Proving statements for all $n$       | Recursion, loop invariants          |
| Relations & functions | Structured pairings                  | Databases, mappings, sorting keys   |

## 1.1 Number Theory

### 1.1.1 Divisibility and the Division Algorithm

**Definition 1.1** (Divisibility). For integers  $a, b$  with  $a \neq 0$ , we say  $a$  **divides**  $b$ , written  $a \mid b$ , if there exists an integer  $k$  with  $b = ak$ . Then  $a$  is a *divisor* (factor) of  $b$  and  $b$  is a *multiple* of  $a$ .

Basic properties (all provable from the definition): if  $a \mid b$  and  $a \mid c$  then  $a \mid (bx + cy)$  for all integers  $x, y$ ; if  $a \mid b$  and  $b \mid c$  then  $a \mid c$  (transitivity).

#### Formula & Mathematical Relation: Division Algorithm

For any integer  $a$  and positive integer  $d$  there exist **unique** integers  $q$  (quotient) and  $r$  (remainder) such that

$$a = dq + r, \quad 0 \leq r < d. \quad (1)$$

We write  $q = a \operatorname{div} d$  and  $r = a \bmod d$ .

#### Common Pitfall & Misconception: Negative Numbers and %

Mathematically  $-7 \bmod 3 = 2$  because  $-7 = 3(-3) + 2$  with  $0 \leq 2 < 3$ . But in C, C++, Java and JavaScript,  $-7 \% 3$  evaluates to  $-1$  (the remainder takes the sign of the dividend). Python's  $-7 \% 3$  gives 2, matching the mathematical definition. This matters in hash tables and circular buffers: always normalise with  $((a \% d) + d) \% d$  in C-family languages.

### 1.1.2 Primes and the Fundamental Theorem of Arithmetic

**Definition 1.2** (Prime). An integer  $p > 1$  is **prime** if its only positive divisors are 1 and  $p$ . An integer  $n > 1$  that is not prime is **composite**.

#### Fundamental Theorem of Arithmetic

Every integer  $n > 1$  can be written as a product of primes, and this factorisation is **unique** up to the order of the factors:

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}.$$

Example:  $360 = 2^3 \cdot 3^2 \cdot 5$ . Consequences: the number of positive divisors of  $n$  is  $(e_1 + 1)(e_2 + 1) \cdots (e_k + 1)$ ; for 360 that is  $4 \cdot 3 \cdot 2 = 24$ .

### 💡 Exam Tip & Key takeaway: Primality Test by Trial Division

If  $n$  is composite, it has a prime factor  $p \leq \sqrt{n}$ . So to test whether  $n$  is prime, divide by every prime up to  $\sqrt{n}$  only. To test  $n = 97$ :  $\sqrt{97} \approx 9.8$ , so check 2, 3, 5, 7. None divide 97, so 97 is prime. Complexity  $O(\sqrt{n})$ —fine for  $n \sim 10^{12}$ , useless for RSA-sized 600-digit numbers, which need probabilistic tests (Miller–Rabin).

There are infinitely many primes (Euclid): if  $p_1, \dots, p_k$  were all of them,  $N = p_1 p_2 \cdots p_k + 1$  is divisible by none of them, contradiction.

### 1.1.3 GCD, LCM and the Euclidean Algorithm

**Definition 1.3.**  $\gcd(a, b)$  is the largest positive integer dividing both  $a$  and  $b$ .  $\text{lcm}(a, b)$  is the smallest positive integer that both divide. Two integers are **coprime** (relatively prime) if  $\gcd(a, b) = 1$ .

### 📊 Formula & Mathematical Relation: Euclidean Algorithm and the GCD–LCM Identity

$$\gcd(a, b) = \gcd(b, a \bmod b), \quad \gcd(a, 0) = a. \quad (2)$$

$$\gcd(a, b) \cdot \text{lcm}(a, b) = a \cdot b. \quad (3)$$

The Euclidean algorithm runs in  $O(\log \min(a, b))$  divisions (Lamé’s theorem: the worst case is consecutive Fibonacci numbers).

### ✏️ Worked Example: Euclidean Algorithm for $\gcd(252, 198)$

$$252 = 1 \cdot 198 + 54$$

$$198 = 3 \cdot 54 + 36$$

$$54 = 1 \cdot 36 + 18$$

$$36 = 2 \cdot 18 + 0$$

The last non-zero remainder is 18, so  $\gcd(252, 198) = 18$  and  $\text{lcm}(252, 198) = \frac{252 \cdot 198}{18} = 2772$ .

### 📖 Bézout’s Identity and the Extended Euclidean Algorithm

For any integers  $a, b$  (not both zero) there exist integers  $x, y$  with

$$ax + by = \gcd(a, b).$$

The **extended Euclidean algorithm** finds  $x, y$  by back-substituting through the Euclidean steps. Continuing the example above:

$$\begin{aligned} 18 &= 54 - 1 \cdot 36 \\ &= 54 - 1 \cdot (198 - 3 \cdot 54) = 4 \cdot 54 - 1 \cdot 198 \\ &= 4 \cdot (252 - 1 \cdot 198) - 1 \cdot 198 = 4 \cdot 252 - 5 \cdot 198. \end{aligned}$$

Check:  $4(252) - 5(198) = 1008 - 990 = 18$ . ✓ So  $x = 4$ ,  $y = -5$ .

### 1.1.4 Modular Arithmetic and Congruences

**Definition 1.4** (Congruence).  $a \equiv b \pmod{m}$  (read “ $a$  is congruent to  $b$  modulo  $m$ ”) iff  $m \mid (a - b)$ , equivalently  $a \bmod m = b \bmod m$ .

Congruence modulo  $m$  is an *equivalence relation* (Section 1.7), so it partitions  $\mathbb{Z}$  into  $m$  **residue classes**  $[0], [1], \dots, [m-1]$ . The set of these classes with addition and multiplication is  $\mathbb{Z}_m$ .

#### Formula & Mathematical Relation: Arithmetic Rules Modulo $m$

If  $a \equiv b$  and  $c \equiv d \pmod{m}$ , then

$$a + c \equiv b + d, \quad a - c \equiv b - d, \quad ac \equiv bd, \quad a^k \equiv b^k \pmod{m}.$$

In computation this means we may reduce mod  $m$  *at every step*, which keeps numbers small:

$$(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m.$$

**Division is NOT generally allowed.**  $a$  has a multiplicative inverse mod  $m$  iff  $\gcd(a, m) = 1$ ; the inverse is the  $x$  from  $ax + my = 1$  (extended Euclid).

#### Worked Example: Modular Inverse of 7 modulo 26 (used in the affine cipher)

We want  $x$  with  $7x \equiv 1 \pmod{26}$ .

$$26 = 3 \cdot 7 + 5$$

$$7 = 1 \cdot 5 + 2$$

$$5 = 2 \cdot 2 + 1$$

Back-substitute:  $1 = 5 - 2 \cdot 2 = 5 - 2(7 - 5) = 3 \cdot 5 - 2 \cdot 7 = 3(26 - 3 \cdot 7) - 2 \cdot 7 = 3 \cdot 26 - 11 \cdot 7$ . So  $7 \cdot (-11) \equiv 1 \pmod{26}$ , and  $-11 \equiv 15$ . Check:  $7 \cdot 15 = 105 = 4 \cdot 26 + 1$ . ✓ Hence  $7^{-1} \equiv 15 \pmod{26}$ .

#### Fermat's Little Theorem, Euler's Totient and Fast Exponentiation

- **Fermat:** if  $p$  is prime and  $p \nmid a$ , then  $a^{p-1} \equiv 1 \pmod{p}$ . Hence  $a^{-1} \equiv a^{p-2} \pmod{p}$ .
- **Euler's totient**  $\phi(n)$  = number of integers in  $\{1, \dots, n\}$  coprime to  $n$ . For a prime  $p$ ,  $\phi(p) = p - 1$ ; for  $n = pq$  with distinct primes,  $\phi(n) = (p - 1)(q - 1)$ ; in general  $\phi(n) = n \prod_{p|n} (1 - 1/p)$ .
- **Euler's theorem:** if  $\gcd(a, n) = 1$  then  $a^{\phi(n)} \equiv 1 \pmod{n}$ . This is the mathematics inside RSA.
- **Fast modular exponentiation** (square-and-multiply) computes  $a^k \bmod m$  in  $O(\log k)$  multiplications. Example:  $3^{13} \bmod 7$ :  $13 = 1101_2$ .  $3^1 = 3$ ,  $3^2 = 9 \equiv 2$ ,  $3^4 \equiv 2^2 = 4$ ,  $3^8 \equiv 4^2 = 16 \equiv 2$ . Then  $3^{13} = 3^8 \cdot 3^4 \cdot 3^1 \equiv 2 \cdot 4 \cdot 3 = 24 \equiv 3 \pmod{7}$ .

#### Worked Example: Toy RSA

Choose primes  $p = 5$ ,  $q = 11$ . Then  $n = 55$ ,  $\phi(n) = 4 \cdot 10 = 40$ . Choose public exponent  $e = 3$  ( $\gcd(3, 40) = 1$ ). Private exponent  $d = e^{-1} \bmod 40$ :  $3 \cdot 27 = 81 = 2 \cdot 40 + 1$ , so  $d = 27$ . Encrypt message  $M = 7$ :  $C = 7^3 \bmod 55 = 343 \bmod 55 = 343 - 6 \cdot 55 = 13$ . Decrypt:  $13^{27} \bmod 55$ . Since  $13^{27} = 13^{16} \cdot 13^8 \cdot 13^2 \cdot 13^1$ :  $13^2 = 169 \equiv 4$ ;  $13^4 \equiv 16$ ;  $13^8 \equiv 256 \equiv 36$ ;  $13^{16} \equiv 36^2 = 1296 \equiv 1296 - 23 \cdot 55 = 31$ . Product:  $31 \cdot 36 = 1116 \equiv 1116 - 20 \cdot 55 = 16$ ;  $16 \cdot 4 = 64 \equiv 9$ ;  $9 \cdot 13 = 117 \equiv 7$ . ✓ We recover  $M = 7$ .



## Where Computer Science Uses This: Hashing and Cryptography

- **Hash tables:** index = key mod  $m$ . Choosing  $m$  prime spreads keys that share common factors; a power of two ignores high-order bits.
- **RSA:** security rests on the difficulty of factoring  $n = pq$ ; correctness rests on Euler's theorem.
- **Linear congruential generators:**  $x_{n+1} = (ax_n + c) \bmod m$  produce pseudo-random numbers; the period is  $m$  only when  $\gcd(c, m) = 1$  and other Hull–Dobell conditions hold.
- **Check digits:** ISBN-10 satisfies  $\sum_{i=1}^{10} i \cdot d_i \equiv 0 \pmod{11}$ .

## 1.2 Set Theory

**Definition 1.5.** A **set** is an unordered collection of distinct objects (*elements*).  $x \in A$  means  $x$  is an element of  $A$ . Sets are described by *roster* ( $A = \{1, 2, 3\}$ ) or *set-builder* notation ( $A = \{x \in \mathbb{Z} : 0 < x < 4\}$ ).

Standard sets:  $\mathbb{N}$  (naturals),  $\mathbb{Z}$  (integers),  $\mathbb{Q}$  (rationals),  $\mathbb{R}$  (reals),  $\emptyset$  (empty set),  $U$  (universal set).



## Subsets, Power Sets and Cardinality

- $A \subseteq B$  (subset) iff every element of  $A$  is in  $B$ .  $A = B$  iff  $A \subseteq B$  and  $B \subseteq A$ —this is how set equality is *proved*.
- $A \subset B$  (proper subset):  $A \subseteq B$  and  $A \neq B$ .  $\emptyset \subseteq A$  for every  $A$ .
- **Power set**  $\mathcal{P}(A)$  = set of all subsets of  $A$ . If  $|A| = n$  then  $|\mathcal{P}(A)| = 2^n$  (each element is either in or out).
- **Cartesian product**  $A \times B = \{(a, b) : a \in A, b \in B\}$ ;  $|A \times B| = |A| \cdot |B|$ . Ordered pairs:  $(1, 2) \neq (2, 1)$ .



## Formula & Mathematical Relation: Set Operations

$$\begin{aligned} A \cup B &= \{x : x \in A \vee x \in B\}, & A \cap B &= \{x : x \in A \wedge x \in B\}, \\ A - B &= \{x : x \in A \wedge x \notin B\}, & \bar{A} &= U - A. \end{aligned}$$

Symmetric difference  $A \oplus B = (A - B) \cup (B - A) = (A \cup B) - (A \cap B)$ .  $A$  and  $B$  are **disjoint** iff  $A \cap B = \emptyset$ .

| Law          | Union form                                       | Intersection form                                |
|--------------|--------------------------------------------------|--------------------------------------------------|
| Identity     | $A \cup \emptyset = A$                           | $A \cap U = A$                                   |
| Domination   | $A \cup U = U$                                   | $A \cap \emptyset = \emptyset$                   |
| Idempotent   | $A \cup A = A$                                   | $A \cap A = A$                                   |
| Complement   | $A \cup \bar{A} = U$                             | $A \cap \bar{A} = \emptyset$                     |
| Commutative  | $A \cup B = B \cup A$                            | $A \cap B = B \cap A$                            |
| Associative  | $(A \cup B) \cup C = A \cup (B \cup C)$          | $(A \cap B) \cap C = A \cap (B \cap C)$          |
| Distributive | $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ | $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ |
| De Morgan    | $\overline{A \cup B} = \bar{A} \cap \bar{B}$     | $\overline{A \cap B} = \bar{A} \cup \bar{B}$     |
| Absorption   | $A \cup (A \cap B) = A$                          | $A \cap (A \cup B) = A$                          |

Table 1: Laws of set algebra. Every law has a twin obtained by swapping  $\cup \leftrightarrow \cap$  and  $\emptyset \leftrightarrow U$  (*duality*). The identical table holds for Boolean algebra and for propositional logic.



### Formula & Mathematical Relation: Principle of Inclusion–Exclusion

$$|A \cup B| = |A| + |B| - |A \cap B| \quad (4)$$

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C| \quad (5)$$



### Worked Example: Inclusion–Exclusion

In a class of 60 students, 35 know Python, 28 know Java and 12 know both. How many know neither?  $|P \cup J| = 35 + 28 - 12 = 51$ , so  $60 - 51 = 9$  students know neither.

Three sets: how many integers in 1..100 are divisible by 2, 3 or 5?  $\lfloor 100/2 \rfloor + \lfloor 100/3 \rfloor + \lfloor 100/5 \rfloor - \lfloor 100/6 \rfloor - \lfloor 100/10 \rfloor - \lfloor 100/15 \rfloor + \lfloor 100/30 \rfloor = 50 + 33 + 20 - 16 - 10 - 6 + 3 = 74$ .



### Exam Tip & Key takeaway: Proving a Set Identity

Two accepted methods:

1. **Element (double inclusion):** take arbitrary  $x$  in LHS, show  $x$  in RHS; then the reverse.
2. **Membership table:** one row per combination of  $x \in A$ ,  $x \in B$ , ... (like a truth table).  
Identical final columns  $\Rightarrow$  equal sets.

Example, De Morgan:  $x \in \overline{A \cup B} \iff \neg(x \in A \vee x \in B) \iff x \notin A \wedge x \notin B \iff x \in \overline{A} \cap \overline{B}$ .



### Where Computer Science Uses This: Sets in Databases and Programming

SQL's UNION, INTERSECT, EXCEPT are exactly  $\cup, \cap, -$ ; a JOIN is a filtered Cartesian product. Python's `set` type implements  $|\&-^$  for  $\cup, \cap, -, \oplus$ . Bitmaps represent subsets of an  $n$ -element universe as  $n$ -bit integers, so  $\cup$  and  $\cap$  become bitwise OR and AND.

## 1.3 Boolean Algebra

**Definition 1.6.** A **Boolean algebra** is a set  $B = \{0, 1\}$  with operations **AND** ( $\cdot$  or  $\wedge$ ), **OR** ( $+$  or  $\vee$ ) and **NOT** ( $\bar{x}$ ,  $x'$  or  $\neg x$ ) satisfying the laws in the table above (with  $\cup \rightarrow +$ ,  $\cap \rightarrow \cdot$ ,  $\emptyset \rightarrow 0$ ,  $U \rightarrow 1$ ).

| $x$ | $y$ | $x \cdot y$ | $x + y$ | $x \oplus y$ | $\bar{x}$ |
|-----|-----|-------------|---------|--------------|-----------|
| 0   | 0   | 0           | 0       | 0            | 1         |
| 0   | 1   | 0           | 1       | 1            | 1         |
| 1   | 0   | 0           | 1       | 1            | 0         |
| 1   | 1   | 1           | 1       | 0            | 0         |



### Key Boolean Laws (the ones you actually use to simplify)

- Complement:  $x + \bar{x} = 1$ ,  $x \cdot \bar{x} = 0$ ,  $\bar{\bar{x}} = x$ .
- Absorption:  $x + xy = x$ ,  $x(x + y) = x$ .
- Adjacency / combining:  $xy + x\bar{y} = x$  (this is what a Karnaugh map does visually).
- Redundancy:  $x + \bar{x}y = x + y$ .
- Consensus:  $xy + \bar{x}z + yz = xy + \bar{x}z$ .
- De Morgan:  $\overline{x + y} = \bar{x}\bar{y}$ ,  $\overline{x\bar{y}} = \bar{x} + y$ .

### Worked Example: Algebraic Simplification

Simplify  $F = \overline{A}B + AB + A\overline{B}$ .

$$\begin{aligned} F &= B(\overline{A} + A) + A\overline{B} && \text{(distributive)} \\ &= B + A\overline{B} && \text{(complement, identity)} \\ &= B + A && \text{(redundancy: } x + \overline{x}y = x + y) \end{aligned}$$

So  $F = A + B$ : a 3-gate circuit collapses to one OR gate.

#### 1.3.1 Canonical Forms

A **minterm** is a product containing every variable exactly once (e.g.  $\overline{A}BC$ ); a **maxterm** is a sum containing every variable once (e.g.  $A + \overline{B} + C$ ). Every function has a unique

- **Sum of Products (SOP)**: OR of the minterms where  $F = 1$ , written  $\sum m(\dots)$ ;
- **Product of Sums (POS)**: AND of the maxterms where  $F = 0$ , written  $\prod M(\dots)$ .

### Worked Example: Truth Table $\rightarrow$ SOP $\rightarrow$ K-map Minimisation

Let  $F(A, B, C) = \sum m(1, 3, 5, 6, 7)$ .

| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

|     |                |   |   |   |
|-----|----------------|---|---|---|
|     | BC=00 01 11 10 |   |   |   |
| A=0 | 0              | 1 | 1 | 0 |
| A=1 | 0              | 1 | 1 | 1 |

Canonical SOP:  $F = \overline{A}\overline{B}C + \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$ . On the K-map (Gray-code column order 00, 01, 11, 10 so adjacent cells differ in one variable), the red  $2 \times 2$  block covers all cells with  $C = 1$ , giving the term  $C$ ; the green pair  $(A\overline{B}\overline{C}, ABC)$  gives  $AB$ . Minimal SOP:  $F = C + AB$ . Rules: group 1s in blocks of 1, 2, 4, 8; blocks may wrap around edges; use the fewest, largest blocks; each 1 must be covered at least once. Don't-care entries ( $\times$ ) may be treated as 1 when it enlarges a block.

### Where Computer Science Uses This: Logic Gates and Conditionals

AND, OR, NOT, NAND, NOR, XOR gates implement these operations in silicon; NAND alone is *functionally complete*. A half-adder computes  $\text{sum} = A \oplus B$  and  $\text{carry} = AB$ . In software, a compiler applies the same laws when it simplifies `if (x && (x || y))` to `if (x)` (absorption).

## 1.4 Logic

### 1.4.1 Propositional Logic

A **proposition** is a declarative sentence that is either true (T) or false (F), never both. Connectives:  $\neg$  (not),  $\wedge$  (and),  $\vee$  (or),  $\rightarrow$  (implies),  $\leftrightarrow$  (iff),  $\oplus$  (xor).

| $p$ | $q$ | $p \wedge q$ | $p \vee q$ | $p \rightarrow q$ | $p \leftrightarrow q$ |
|-----|-----|--------------|------------|-------------------|-----------------------|
| T   | T   | T            | T          | T                 | T                     |
| T   | F   | F            | T          | <b>F</b>          | F                     |
| F   | T   | F            | T          | T                 | F                     |
| F   | F   | F            | F          | T                 | T                     |

### ⚠ Common Pitfall & Misconception: The Implication Row Everyone Gets Wrong

$p \rightarrow q$  is false in **exactly one** case:  $p$  true and  $q$  false. When the hypothesis  $p$  is false the implication is *vacuously true*. “If it rains, I carry an umbrella” is not violated on a dry day. Also:  $p \rightarrow q \equiv \neg p \vee q$  (memorise—it converts implications into something you can manipulate algebraically).

### 📖 Converse, Inverse, Contrapositive

For the implication  $p \rightarrow q$ :

|                |                             |                                                                |
|----------------|-----------------------------|----------------------------------------------------------------|
| Converse       | $q \rightarrow p$           | <b>not</b> equivalent to original                              |
| Inverse        | $\neg p \rightarrow \neg q$ | <b>not</b> equivalent to original (but equivalent to converse) |
| Contrapositive | $\neg q \rightarrow \neg p$ | <b>equivalent</b> to original                                  |

Proof by contrapositive: to show “if  $n^2$  is even then  $n$  is even”, prove “if  $n$  is odd then  $n^2$  is odd”:  $n = 2k + 1 \Rightarrow n^2 = 4k^2 + 4k + 1$ , odd.

**Definition 1.7.** A **tautology** is true for every assignment (e.g.  $p \vee \neg p$ ); a **contradiction** is false for every assignment ( $p \wedge \neg p$ ); otherwise the formula is a **contingency**. Two formulas are **logically equivalent** ( $\equiv$ ) if they have identical truth tables, i.e.  $P \leftrightarrow Q$  is a tautology.

### ✍ Worked Example: Proving an Equivalence Without a Truth Table

Show  $(p \rightarrow q) \wedge (p \rightarrow r) \equiv p \rightarrow (q \wedge r)$ .

$$\begin{aligned}
 (p \rightarrow q) \wedge (p \rightarrow r) &\equiv (\neg p \vee q) \wedge (\neg p \vee r) && \text{(implication law)} \\
 &\equiv \neg p \vee (q \wedge r) && \text{(distributive)} \\
 &\equiv p \rightarrow (q \wedge r) && \text{(implication law)}
 \end{aligned}$$

## 1.4.2 Rules of Inference

| Rule                   | Form                                                          | Tautology                                              |
|------------------------|---------------------------------------------------------------|--------------------------------------------------------|
| Modus ponens           | $p, p \rightarrow q \therefore q$                             | $(p \wedge (p \rightarrow q)) \rightarrow q$           |
| Modus tollens          | $\neg q, p \rightarrow q \therefore \neg p$                   | $(\neg q \wedge (p \rightarrow q)) \rightarrow \neg p$ |
| Hypothetical syllogism | $p \rightarrow q, q \rightarrow r \therefore p \rightarrow r$ |                                                        |
| Disjunctive syllogism  | $p \vee q, \neg p \therefore q$                               |                                                        |
| Addition               | $p \therefore p \vee q$                                       |                                                        |
| Simplification         | $p \wedge q \therefore p$                                     |                                                        |
| Resolution             | $p \vee q, \neg p \vee r \therefore q \vee r$                 | (basis of SAT solvers, Unit V)                         |

Table 2: Rules of inference. The two invalid look-alikes: *affirming the consequent* ( $q, p \rightarrow q \therefore p$ ) and *denying the antecedent* ( $\neg p, p \rightarrow q \therefore \neg q$ ) are fallacies.

## 1.4.3 Predicate Logic and Quantifiers

A **predicate**  $P(x)$  becomes a proposition once  $x$  is assigned a value from the *domain*. Quantifiers:  $\forall x P(x)$  (“for all”) and  $\exists x P(x)$  (“there exists”).

### Formula & Mathematical Relation: Negating Quantifiers (De Morgan for quantifiers)

$$\neg \forall x P(x) \equiv \exists x \neg P(x), \quad \neg \exists x P(x) \equiv \forall x \neg P(x).$$

Order matters for nested quantifiers:  $\forall x \exists y (x < y)$  is true over  $\mathbb{Z}$ , but  $\exists y \forall x (x < y)$  is false.

### Exam Tip & Key takeaway: Translate, Then Negate

“Every student has submitted some assignment”:  $\forall s \exists a \text{Sub}(s, a)$ . Negation:  $\exists s \forall a \neg \text{Sub}(s, a)$  — “some student has submitted no assignment.” Push the negation inward one quantifier at a time, flipping each.

### Where Computer Science Uses This: Logic in Software

SQL **WHERE** clauses are propositional formulas; **NOT EXISTS** is  $\neg \exists$ . Loop invariants and Hoare triples  $\{P\} C \{Q\}$  are predicate-logic statements about program state. Prolog and expert systems run resolution. Type checkers, model checkers and SAT/SMT solvers (Unit V) are inference engines.

## 1.5 Calculus for Computer Science

Calculus enters CS through **optimisation** (every ML model is trained by minimising a loss), **continuous modelling** (physics engines, signal processing) and **analysis of algorithms** (sums approximated by integrals).

### Limits and Continuity

$\lim_{x \rightarrow a} f(x) = L$  means  $f(x)$  can be made arbitrarily close to  $L$  by taking  $x$  close enough to  $a$ .  $f$  is continuous at  $a$  if  $\lim_{x \rightarrow a} f(x) = f(a)$ . Important limits:  $\lim_{x \rightarrow 0} \frac{\sin x}{x} = 1$ ,  $\lim_{n \rightarrow \infty} (1 + 1/n)^n = e$ ,  $\lim_{n \rightarrow \infty} \frac{\log n}{n} = 0$  (the last one is why  $O(\log n) \subset O(n)$  in Unit V).

### Formula & Mathematical Relation: Derivatives You Must Know

$$\frac{d}{dx} x^n = nx^{n-1}, \quad \frac{d}{dx} e^x = e^x, \quad \frac{d}{dx} \ln x = \frac{1}{x}, \quad \frac{d}{dx} \sin x = \cos x, \quad \frac{d}{dx} \cos x = -\sin x.$$

Product rule  $(uv)' = u'v + uv'$ ; quotient rule  $(u/v)' = (u'v - uv')/v^2$ ; **chain rule**  $\frac{d}{dx} f(g(x)) = f'(g(x)) g'(x)$ .

Sigmoid  $\sigma(x) = \frac{1}{1 + e^{-x}}$  has the derivative  $\sigma'(x) = \sigma(x)(1 - \sigma(x))$  — the workhorse of logistic regression and neural networks.

### Partial Derivatives and the Gradient

For  $f(x, y)$ , the partial derivative  $\partial f / \partial x$  differentiates with respect to  $x$  treating  $y$  as constant. The **gradient**

$$\nabla f = \left( \frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right)$$

points in the direction of *steepest increase*. Hence **gradient descent**:  $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)$  walks downhill. Critical points satisfy  $\nabla f = 0$ ; the second derivative (Hessian) decides min/max/saddle.

### Worked Example: Minimising a Loss Function

Let  $L(w) = (3w-6)^2 + w^2$ . Then  $L'(w) = 2(3w-6) \cdot 3 + 2w = 18w - 36 + 2w = 20w - 36$ . Setting  $L'(w) = 0$  gives  $w^* = 1.8$ .  $L''(w) = 20 > 0$  so it is a minimum;  $L(1.8) = (5.4 - 6)^2 + 3.24 = 0.36 + 3.24 = 3.6$ .

Gradient descent from  $w_0 = 0$  with  $\alpha = 0.02$ :  $w_1 = 0 - 0.02(-36) = 0.72$ ;  $w_2 = 0.72 - 0.02(20 \cdot 0.72 - 36) = 0.72 + 0.432 = 1.152$ ;  $w_3 = 1.152 + 0.02(36 - 23.04) = 1.4112$  — converging toward 1.8.

### Formula & Mathematical Relation: Integration

$\int x^n dx = \frac{x^{n+1}}{n+1} + C$  ( $n \neq -1$ ),  $\int \frac{1}{x} dx = \ln|x| + C$ ,  $\int e^x dx = e^x + C$ . Fundamental theorem:

$\int_a^b f(x) dx = F(b) - F(a)$  where  $F' = f$ .

**Sum-integral bound used in algorithm analysis:** for increasing  $f$ ,  $\int_0^n f(x) dx \leq \sum_{k=1}^n f(k) \leq \int_1^{n+1} f(x) dx$ . Hence  $\sum_{k=1}^n \frac{1}{k} = \Theta(\ln n)$  (harmonic number  $H_n$ ) and  $\sum_{k=1}^n \log k = \Theta(n \log n)$  (the lower bound for comparison sorting).

### Where Computer Science Uses This: Calculus in Practice

Backpropagation is the chain rule applied layer by layer. Expected value  $E[X] = \int x f(x) dx$  (Unit III). The area under an ROC curve is an integral. Physics engines integrate velocity to get position. Learning-rate schedules are chosen by looking at the second derivative (curvature) of the loss.

## 1.6 Mathematical Induction

### Principle of Mathematical Induction

To prove  $P(n)$  for all integers  $n \geq n_0$ :

1. **Basis step:** prove  $P(n_0)$ .
2. **Inductive step:** assume  $P(k)$  holds for an arbitrary  $k \geq n_0$  (the *inductive hypothesis*), and prove  $P(k+1)$ .

Then  $P(n)$  is true for all  $n \geq n_0$ . Think dominoes: the first falls, and each falling domino knocks over the next.

**Strong induction** assumes  $P(n_0), \dots, P(k)$  all hold when proving  $P(k+1)$ . It is needed when  $P(k+1)$  depends on cases smaller than  $k$  (e.g. factorisation into primes, recurrences like Fibonacci, divide-and-conquer algorithms).

### Worked Example: Summation Formula

Prove  $1 + 2 + \dots + n = \frac{n(n+1)}{2}$  for all  $n \geq 1$ .

*Basis:*  $n = 1$ : LHS = 1, RHS =  $\frac{1 \cdot 2}{2} = 1$ . ✓

*Inductive step:* assume  $1 + \dots + k = \frac{k(k+1)}{2}$ . Then

$$1 + \dots + k + (k+1) = \frac{k(k+1)}{2} + (k+1) = \frac{(k+1)(k+2)}{2},$$

which is the formula at  $n = k+1$ . By induction the statement holds for all  $n \geq 1$ . ■

### Worked Example: Divisibility

Prove  $7 \mid (8^n - 1)$  for all  $n \geq 0$ .

*Basis:*  $8^0 - 1 = 0 = 7 \cdot 0$ . ✓

*Inductive step:* assume  $8^k - 1 = 7m$  for some integer  $m$ . Then

$$8^{k+1} - 1 = 8 \cdot 8^k - 1 = 8(8^k - 1) + 7 = 8 \cdot 7m + 7 = 7(8m + 1),$$

divisible by 7. ■

### Worked Example: Inequality (the kind that appears in complexity proofs)

Prove  $2^n > n^2$  for all  $n \geq 5$ .

*Basis:*  $2^5 = 32 > 25$ . ✓

*Inductive step:* assume  $2^k > k^2$  with  $k \geq 5$ . Then  $2^{k+1} = 2 \cdot 2^k > 2k^2$ . It remains to show  $2k^2 \geq (k+1)^2 = k^2 + 2k + 1$ , i.e.  $k^2 - 2k - 1 \geq 0$ , i.e.  $(k-1)^2 \geq 2$ , true for  $k \geq 3$ . Hence  $2^{k+1} > (k+1)^2$ . ■

### Worked Example: Strong Induction: Postage Stamps

Every amount  $n \geq 12$  can be formed with 4-cent and 5-cent stamps.

*Basis:*  $12 = 4 + 4 + 4$ ,  $13 = 4 + 4 + 5$ ,  $14 = 4 + 5 + 5$ ,  $15 = 5 + 5 + 5$ .

*Inductive step:* for  $k+1 \geq 16$ , by the strong hypothesis  $(k+1) - 4 \geq 12$  can be formed; add one 4-cent stamp. ■ (Four base cases are needed because the step reaches back by 4.)

### Common Pitfall & Misconception: Two Classic Induction Errors

- **Forgetting the basis step.** “ $n = n + 1$ ” has a perfectly valid inductive step and is false.
- **A step that fails for small  $k$ .** The famous “all horses are the same colour” proof breaks at  $k = 1 \rightarrow 2$  because the two overlapping subsets are disjoint there. Always check the inductive step works at the *first*  $k$ .

### Where Computer Science Uses This: Induction Proves Programs Correct

**Loop invariants** are induction on the iteration count: invariant holds before the loop (basis), each iteration preserves it (step), so it holds at exit. **Recursive algorithms** are proved by strong induction on input size: merge sort is correct on size  $n$  because it is correct on the two halves. Every recurrence solution in Unit V is verified by induction (the *substitution method*).

## 1.7 Relations and Functions

### 1.7.1 Relations

**Definition 1.8.** A (binary) **relation**  $R$  from  $A$  to  $B$  is a subset of  $A \times B$ . We write  $a R b$  for  $(a, b) \in R$ . A relation *on*  $A$  is a subset of  $A \times A$ . It can be pictured as a directed graph (an arrow from  $a$  to  $b$  when  $a R b$ ) or as a 0/1 matrix  $M_R$  with  $M_{ij} = 1$  iff  $a_i R a_j$ .

### Equivalence Relations and Partial Orders

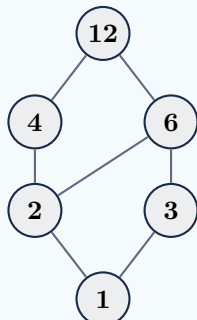
- **Equivalence relation** = reflexive + symmetric + transitive. It partitions the set into disjoint **equivalence classes**. Example:  $a \equiv b \pmod{3}$  splits  $\mathbb{Z}$  into  $[0], [1], [2]$ . Conversely every partition defines an equivalence relation. (This is the mathematical content of “group by” in SQL and of union-find.)

| Property      | Definition                         | Matrix / digraph view                                | Example on $\mathbb{Z}$                         |
|---------------|------------------------------------|------------------------------------------------------|-------------------------------------------------|
| Reflexive     | $\forall a : aRa$                  | all diagonal entries 1; loop at every vertex         | $\leq, =$ , “divides”                           |
| Irreflexive   | $\forall a : \neg aRa$             | all diagonal entries 0                               | $<$                                             |
| Symmetric     | $aRb \Rightarrow bRa$              | $M = M^T$ ; every arrow has a reverse                | $=, \equiv \pmod{m}$                            |
| Antisymmetric | $aRb \wedge bRa \Rightarrow a = b$ | no pair of opposite arrows between distinct vertices | $\leq, \subseteq$ , “divides” on $\mathbb{Z}^+$ |
| Transitive    | $aRb \wedge bRc \Rightarrow aRc$   | every 2-step path has a shortcut                     | $\leq, <, =$                                    |

Table 3: Properties of relations. “Symmetric” and “antisymmetric” are not opposites:  $=$  is both, and  $\{(1, 2), (2, 1), (1, 3)\}$  is neither.

- **Partial order** = reflexive + antisymmetric + transitive; the set is then a *poset*. Examples:  $(\mathbb{Z}, \leq)$ ,  $(\mathcal{P}(S), \subseteq)$ ,  $(\mathbb{Z}^+, |)$ . Elements  $a, b$  are *comparable* if  $a \preceq b$  or  $b \preceq a$ ; a partial order where all pairs are comparable is a **total order** (e.g.  $\leq$ , but not  $\subseteq$  or  $|$ ).
- A **Hasse diagram** draws a poset with transitive and reflexive edges removed and larger elements above smaller ones.

#### Worked Example: Hasse Diagram of $D_{12} = \{1, 2, 3, 4, 6, 12\}$ under Divisibility



Minimal and least element: 1. Maximal and greatest: 12. 4 and 6 are incomparable (neither divides the other), so this is not a total order. The greatest lower bound of 4 and 6 is  $\gcd = 2$ ; the least upper bound is  $\text{lcm} = 12$  — divisibility posets are *lattices*.

#### Composition and Closure

$S \circ R = \{(a, c) : \exists b, (a, b) \in R \wedge (b, c) \in S\}$ ; in matrix terms  $M_{S \circ R} = M_R \odot M_S$  (Boolean product).  $R^n = n$ -fold composition. The **transitive closure**  $R^* = R \cup R^2 \cup \dots \cup R^{|A|}$  is the smallest transitive relation containing  $R$ ; Warshall’s algorithm computes it in  $O(n^3)$ . In a graph,  $(a, b) \in R^*$  iff there is a path from  $a$  to  $b$  (Unit IV).

### 1.7.2 Functions

**Definition 1.9.** A **function**  $f : A \rightarrow B$  is a relation that assigns to *every*  $a \in A$  *exactly one*  $b \in B$ , written  $f(a) = b$ .  $A$  is the **domain**,  $B$  the **codomain**, and  $f(A) = \{f(a) : a \in A\} \subseteq B$  the **range** (image).

## Injective, Surjective, Bijective

- **Injective (one-to-one):**  $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$ . Distinct inputs, distinct outputs. Test: a horizontal line meets the graph at most once.
- **Surjective (onto):** range = codomain; every  $b$  is hit by some  $a$ .
- **Bijective:** both. A bijection has an **inverse**  $f^{-1} : B \rightarrow A$  with  $f^{-1}(f(a)) = a$ . For finite sets, a bijection  $A \rightarrow B$  exists iff  $|A| = |B|$  (this is how we *count* by matching, and why a hash function on  $n$  keys into fewer than  $n$  slots *must* collide—the pigeonhole principle).

Examples on  $\mathbb{R}$ :  $f(x) = 2x + 1$  bijective;  $f(x) = x^2$  neither (not injective:  $f(-2) = f(2)$ ; not surjective: never negative);  $f(x) = e^x$  injective, not surjective onto  $\mathbb{R}$  but bijective onto  $(0, \infty)$ .

## Formula & Mathematical Relation: Composition, Inverse, Floor and Ceiling

$(g \circ f)(x) = g(f(x))$  — apply  $f$  first. Composition is associative but not commutative. If  $f$  and  $g$  are bijections then  $(g \circ f)^{-1} = f^{-1} \circ g^{-1}$ .

$\lfloor x \rfloor$  = greatest integer  $\leq x$ ;  $\lceil x \rceil$  = least integer  $\geq x$ .  $\lfloor 3.7 \rfloor = 3$ ,  $\lfloor -3.7 \rfloor = -4$ ,  $\lceil 3.2 \rceil = 4$ . Useful:  $\lceil n/2 \rceil + \lfloor n/2 \rfloor = n$ ; the number of bits in  $n$  is  $\lfloor \log_2 n \rfloor + 1$ ; a binary search on  $n$  items makes at most  $\lceil \log_2(n+1) \rceil$  probes.

## Worked Example: Classifying a Function

Let  $f : \mathbb{Z} \rightarrow \mathbb{Z}$ ,  $f(n) = 2n + 3$ . Injective:  $2a + 3 = 2b + 3 \Rightarrow a = b$ . ✓ Surjective?  $f(n) = 4$  requires  $n = 1/2 \notin \mathbb{Z}$ , so no. Not bijective on  $\mathbb{Z}$ . The same rule  $\mathbb{R} \rightarrow \mathbb{R}$  is bijective with  $f^{-1}(y) = (y - 3)/2$ .

Let  $g(n) = n \bmod 5$  on  $\mathbb{Z} \rightarrow \{0, 1, 2, 3, 4\}$ . Surjective (every residue occurs) but not injective ( $g(0) = g(5)$ ). This is every hash function: onto a small table, many-to-one.

## Where Computer Science Uses This: Relations and Functions in Systems

A database table *is* a relation (a subset of a Cartesian product of column domains). A primary key is an injective function from rows to key values. Foreign keys are relations between tables. Sorting requires a total order on keys; **Comparable** in Java is a contract that the comparison is a total order. A **pure function** in programming is exactly a mathematical function: same input, same output, no side effects.

## 1.8 Unit I Quick-Revision Sheet

### Exam Tip & Key takeaway: Ten Things to Carry Into the Exam

1.  $a = dq + r$ ,  $0 \leq r < d$ ;  $\gcd(a, b) = \gcd(b, a \bmod b)$ ;  $\gcd \cdot \text{lcm} = ab$ .
2.  $a^{-1} \bmod m$  exists iff  $\gcd(a, m) = 1$ ; find it with extended Euclid. Fermat:  $a^{p-1} \equiv 1 \pmod{p}$ .
3.  $|\mathcal{P}(A)| = 2^{|A|}$ ;  $|A \cup B| = |A| + |B| - |A \cap B|$ .
4. De Morgan in three costumes:  $\overline{A \cup B} = \overline{A} \cap \overline{B}$ ;  $\overline{x + y} = \overline{x} \overline{y}$ ;  $\neg(p \vee q) \equiv \neg p \wedge \neg q$ ;  $\neg \forall x P \equiv \exists x \neg P$ .
5.  $p \rightarrow q \equiv \neg p \vee q \equiv \neg q \rightarrow \neg p$  (contrapositive). Converse and inverse are *not* equivalent to it.
6. K-map: Gray-code order, group in powers of two, wrap around edges.
7.  $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ ; gradient points uphill, so descend with  $-\alpha \nabla f$ .
8. Induction = basis + inductive step; use *strong* induction when the step reaches back more than one.
9. Equivalence = R+S+T (partition); partial order = R+A+T (Hasse diagram).

10. Bijection  $\iff$  inverse exists; finite  $A \rightarrow B$  bijection  $\iff |A| = |B|$ .